Introduction

- ✓ Express.js is a small framework that works on top of Node.js web server functionality to simplify its APIs and add helpful new features.
- ✓ It makes it easier to organize your application's functionality with middleware and routing.
- ✓ It adds helpful utilities to Node.js HTTP objects and facilitates the rendering of dynamic HTTP objects.

Features of Express JS

- ✓ Develops Node.js web applications quickly and easily.
- ✓ It's simple to set up and personalize.
- ✓ Allows you to define application routes using HTTP methods and URLs.
- ✓ Includes a number of middleware modules that can be used to execute additional requests and responses activities.
- ✓ Simple to interface with a variety of template engines, including pug(Jade), EJS.
- ✓ Allows you to specify a middleware for handling errors.

Environment Setup

Node Package Manager(npm)

- ✓ **npm** is the package manager for node.
- ✓ The **npm** Registry is a public collection of packages of open-source code for Node.js, front-end web apps, mobile apps, robots, routers, and countless other needs of the JavaScript community.
- ✓ **npm** allows us to access all these packages and install them locally. You can browse through the list of packages available on npm at npmJS.

How to use npm?

There are two ways to install a package using npm: globally and locally.

- **Globally** – This method is generally used to install development tools and CLI based packages. To install a package globally, use the following code.

```
npm install -g <package-name>
```

- **Locally** – This method is generally used to install frameworks and libraries. A locally installed package can be used only within the directory it is installed. To install a package locally, use the same command as above without the **-g** flag.

npm install <package-name>

Whenever we create a project using npm, we need to provide a **package.json** file, which has all the details about our project. npm makes it easy for us to set up this file. Let us set up our development project.

Step 1 – Start your terminal/cmd, create a new folder named hello-world and cd (create directory) into it

```
mkdir express
cd express
```

Step 2 – Now to create the package.json file using npm, use the following code.

```
npm init
```

It will ask you for the following information.

```
{
  "name": "new-folder",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
Is this OK? (yes)
```

Just keep pressing enter, and enter your name at the "author name" field.

Set proxy if required: npm config set proxy http://192.168.10.251:808

Step 3 – Now we have our package.json file set up, we will further install Express. To install Express and add it to our package.json file, use the following command –

npm install express --save

The --**save** flag can be replaced by the **-S** flag. This flag ensures that Express is added as a dependency to our **package.json** file. This has an advantage, the next time we need to install all the dependencies of our project we can just run the command *npm install* and it will find the dependencies in this file and install them for us.

We can also install express by below command.

npm install express
OR
npm i express

<u>To check version</u> of express run below command or you can check in package.json file.

npm list express

This is all we need to start development using the Express framework.

Create a new file called index.js and type the following in it. (You can create file with any name e1.js,test.js etc)

Index.js

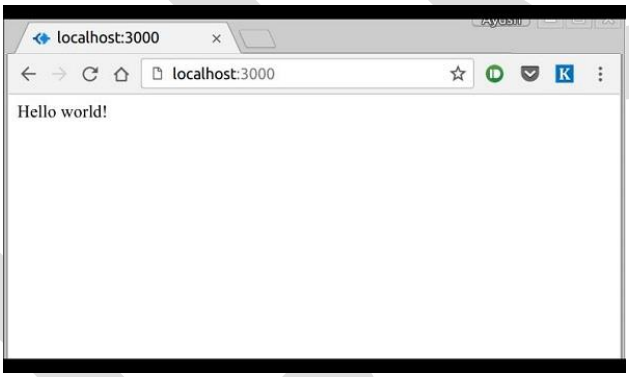
```
var express = require('express');
var app = express();
app.get('/', function(req, res){
 res.set ("content-type","text/plain");
 res.send("Hello world!");

});
app.listen(3000);
```

Save the file, go to your terminal and type the following.

```
node index.js
```

This will start the server. To test this app, open your browser and go to **http://localhost:3000** and a message will be displayed as in the following screenshot.



How the App Works?

- ✓ The first line imports Express in our file, we have access to it through the variable Express.
- ✓ We use it to create an application and assign it to var app.

```
const express = require('express');

const app = express();
```

- ✓ Importing the Express module.
- ✓ express() is a function that creates an instance of an Express application.

- ✓ Creating an app instance (like creating a server) that you can use to define routes, middleware, etc.

app.get(route, callback)

- ✓ This function tells what to do when a **get** request at the given route is called. The callback function has 2 parameters, ***request(req)*** and ***response(res)***.
- ✓ The request **object(req)** represents the HTTP request and has properties for the request query string, parameters, body, HTTP headers, etc. Similarly, the response object represents the HTTP response that the Express app sends when it receives an HTTP request.

res.set()

- ✓ The res.set() function is used to set the response HTTP header field to value. To set multiple fields at once, pass an object as the parameter.

✓ **res.set(field [, value])**

- ✓ **Parameters**: The field parameter is the name of the field and the value parameter is the value assigned to the field parameter.
- ✓ **Return Value**: It returns an Object.

Method	Code Example	Use Case
res.set()	**res.set("content-type", "text/plain")**	**Manually set headers**
res.type()	res.type("text/plain")	Set content type
res.setHeader()	res.setHeader("Content-Type", "text/plain")	Node.js style

res.send()

- ✓ This function takes an object as input and it sends this to the requesting client. Here we are sending the string *"Hello World!"*.

app.listen(port, [host], [backlog], [callback]])

- ✓ This function binds and listens for connections on the specified host and port. Port is the only required parameter here.

Sr. No.	Argument & Description
1	**port** A port number on which the server should accept incoming requests.
2	**host** Name of the domain. You need to set it when you deploy your apps to the cloud.
3	**backlog** The maximum number of queued pending connections. The default is 511.
4	**callback** An asynchronous function that is called when the server starts listening for requests.

```
app.listen(3000, 'localhost', 100, () => {console.log('Server started');});
```

Response methods

The methods on the response object (res) in the following table can send a response to the client, and terminate the request-response cycle.

Method	Description	Example	When to Use
res.send()	Sends response (text, HTML, JSON, etc.)	res.send("Hello")	General purpose
res.json()	Sends JSON response	res.json({ name: "Rahul" })	APIs
res.end()	res.end() is used to end the response. It can optionally send data, but it does not automatically set headers like Express methods such as res.send() or res.json().	res.end()	Minimal response handling
res.sendFile()	Sends a file	res.sendFile("file.txt")	File downloads or serving
res.redirect()	Redirects client to another URL	res.redirect("/home")	Navigation or routing
res.render()	Renders a template (view engine required)	res.render("index")	Server-side rendering

- ✓ **res.send:** Sends data and end the request.
- ✓ **res.json:** Sends data in JSON format and ends the request. **res.json()** allows for extra formatting of the JSON data - if this is not required **res.send()** can also be used to return a response object using Express. Both of these methods also end the response correctly, and there's no further action required.

app.use()

What is app.use()

- In Express.js, app.use() is used to attach middleware functions to your application.
- A middleware is a function that runs during the request and response cycle.
- Flow of execution:
- Request → Middleware → Route → Response

When do we use app.use()

Use app.use() when you want something to run:

- Before every request
- Before specific routes
- For processing incoming data
- For adding features such as security, logging, or validation

Syntax

```
app.use(path, middlewareFunction)
```

- If no path is provided, it runs for all routes
- If a path is provided, it runs only for matching routes

Use Case	Code	Why it is used	When to use
JSON	express.json()	Read JSON body	APIs
URL Encoded	express.urlencoded()	Read form data	Forms
CORS	cors()	Allow cross-origin	Frontend and Backend
Static Files	express.static()	Serve files	Websites
Custom	(req, res, next) function	Custom logic	Logging, Authentication

JSON Middleware

```
app.use(express.json())
Used to parse JSON data from incoming requests and store it in req.body.
```

URL Encoded Middleware

```
app.use(express.urlencoded({ extended: true }))
```

Used to parse form data sent from HTML forms.

CORS Middleware

app.use(cors())
Used to allow communication between frontend and backend running on different domains or ports.

Static Files Middleware

app.use(express.static('public'))
Used to serve static files like HTML, CSS, JavaScript, and images from a folder.

Custom Middleware

```
app.use((req, res, next) => {
console.log(req.method, req.url);
next();
})
```

Used to add custom logic such as logging, authentication, or validation.

Example (For reference only)

```
const express = require('express');
const cors = require('cors');

const app = express();

// Middleware usage
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(cors());
app.use(express.static('public'));

// Custom middleware
app.use((req, res, next) => {
console.log(req.method, req.url);
next();
});

// Start server
app.listen(3000);
```

NOTE:

This app.use() is currently for understanding purposes. These middleware functions will be used later based on project requirements. Not all middleware is required in every application, and they should be applied only when needed.

Routing

Introduction to Routing

- ✓ In Express.js, routing refers to the process of determining how an application responds to a client request for a specific endpoint.
- ✓ An endpoint is defined by a URI (Uniform Resource Identifier) or path along with an HTTP request method such as GET, POST, PUT, or DELETE.
- ✓ Routing is one of the core features of Express.js and is used to build APIs and web applications.
- ✓ Each route in Express can have one or more handler functions. These handler functions are executed when a request matches the defined route.

Difference between URL and URI

A URL (Uniform Resource Locator) is used to specify the location of a resource on the internet. It provides information about where the resource is located and how to access it. A URI (Uniform Resource Identifier) is a more general concept that is used to identify any resource, not only those available on the internet. A URL is a type of URI.

Example:

API in Express.js

Consider an API endpoint in Express.js

GET /users/101

Here, /users/101 is a **URI** because it identifies a specific user resource.

If this endpoint is accessed using

http://localhost:3000/users/101

then it becomes a **URL** because it includes the protocol and location needed to access the resource.

Structure of Route Definition

Route definition takes the following structure:

```
app.METHOD(PATH, HANDLER)
```

- **app** is an instance of express.
- **METHOD** is an HTTP request method, in lowercase.
- **PATH** is a route path on the server.
- **HANDLER** is the callback function that executes when the route is matched.

Routing using HTTP Methods

HTTP methods define the type of operation that should be performed on a resource. The most commonly used HTTP methods in Express.js are GET, POST, PUT, and DELETE.

- ✓ GET is used to retrieve data from the server.
- ✓ POST is used to send data to the server.
- ✓ PUT is used to update existing data.
- ✓ DELETE is used to remove data from the server.

Example of a GET request on the homepage:

```
app.get('/', (req, res) => {
res.send('Hello World!')
})
```

In this example, when a client sends a GET request to the root URL, the server responds with "Hello World".

Example of a POST request on the homepage:

```
app.post('/', (req, res) => {
res.send('Got a POST request')
})
```

In this case, when a POST request is made to the same path, a different response is sent. These examples show that the same route can behave differently depending on the HTTP method used.

Handling Multiple HTTP Methods using app.all()

The app.all() function in Express.js is used to handle all types of HTTP requests for a specific route. This means it will respond to GET, POST, PUT, DELETE, and any other HTTP method.

```
Example:
app.all('/user', (req, res) => {
res.send('This route handles all HTTP methods')
})
```

Using app.all() helps reduce code duplication when the same logic needs to be applied to multiple request methods.

Key Points about Routing and HTTP Methods

- ✓ Routing connects client requests to server responses.
- ✓ Each route is defined using a method, path, and handler function.
- ✓ Different HTTP methods can be used on the same route to perform different operations.
- ✓ The app.all() method allows handling all HTTP methods with a single route definition.
- ✓ Routing is essential for building RESTful APIs and web applications.

Example

1.js

```
const express = require("express");
const app = express();
app.get ("/", (req,res)=>
{
   res.set ("content-type","text/plain");
   res.send ("<h1>Hello</h1>");
});
app.get ("/about", (req,res)=>
{
   res.set ("content-type","text/html");
   res.write ("Hello");
   res.send ("<h1> Hello from home</h1>");
   //res.write ("hello");
});
app.listen (5504,()=>
{
   console.log ("server started");
})
```

Output

In terminal > node 1.js

Console >>

server started

In browser >>

- **localhost:5504**
  <h1>Hello</h1>
- **localhost:5504/about**
  Hello

Note: Like node JS we cannot set headers after they are sent to the client. Means if we send response by writing in "res.write()" then "res.send()" must be blank.

*Here in example on about page only nothing will be displayed. And in console it will give an error "Cannot set headers after they are sent to the client". Means we cannot write anything in **res.send()**.*

***res.write()** after the res.send() will generate an error "write after end".(Commented line)*

JSON Processing

We can define a JSON object and directly pass it inside res.send(). For this, JSON.stringify() method is not required. To send JSON object in res.write(), convert JSON object to string using JSON.stringify().

Example : Write Express JS script to request server to display json object on browser.

Send JSON object using res.write	**Send JSON object using res.send**

const express=require("express")
const app=express();
student={name:"LJU",age:28}
app.get("/",(req,res)=>{
res.write(JSON.stringify(student))
res.send()
})
app.listen(6007)
	const express=require("express")
	const app=express();
	student={name:"LJU",age:28}
	app.get("/",(req,res)=>{
	res.send(student)
	})
	app.listen(6007)

Output in browser: {"name":"LJU","age":28}	**Output in browser:** {"name":"LJU","age":28}

Write express script to send json object to server and display it.

Send JSON object using res.json

```
const express=require("express")
const app=express();
student={name:"LJU",age:28}
  app.get("/",(req,res)=>{
    res.json(student)
  })
app.listen(6007)
```

Output in browser: {"name":"LJU","age":28}

Example : Write Express JS script to request server to display only Age on browser.

```
const express=require("express")
const app=express();
student={name:"LJU",age:28}
   app.get("/",(req,res)=>{
      res.write(student.age+"")   //needs to convert to string so appended string
      res.send()
   })
   app.get("/j",(req,res)=>{
      res.send(student.age)
   })
   app.get("/j1",(req,res)=>{
      res.json(student.age)
   })
app.listen(6001)
```

Example : Write Express JS script to request server to display json object (Array of Objects) in table form on browser.

```
const express=require("express")
const app=express();
student={
   u1:[{name:"LJU",id:2},
   {name:"LJU1",id:3},
   {name:"LJU2",id:4},
   {name:"LJU3",id:5},
   {name:"LJU4",id:6}]}
      app.get("/student",(req,res)=>{
        res.set("content-type","text/html")
              res.write("<center><table   cellspacing='5px'
cellpadding='8px'                                  border='1px
solid'><tr><th>Name</th><th>ID</th></tr>")
         for(i of student.u1){
            res.write("<tr><td>" + i.name + "</td>")
            res.write("<td>" + i.id + "</td></tr>")
         }
        res.write("</table></center>")
        res.send()
      })
app.listen(6007)
```

Output:

Name	ID
LJU	2
LJU1	3
LJU2	4
LJU3	5
LJU4	6

Example: Write an express js script to define one JSON array of 3 objects having properties name and age. Sort this object according to age. If user requests sorted names in url then all names along with age should be printed according to descending order of age. Also, display these sorted values on "Sort page" and display JSON object on "Home page"

```
const express = require("express")
const app = express();
student = [{name:"abc",age:28},
        {name:"def",age:40},
        {name:"xyz",age:50}]
 //home page
app.get ("/",(req,res)=>{
   res.send (student)
})
//sort page
app.get ("/sort",(req,res)=>{
   res.set ("content-type","text/html")
   var des = student.sort((a,b)=>b.age-a.age)
   for (k of des){
      res.write ("<center><h2>"+k.name+ " = " +k.age +"</h2></center>")
   }
   res.send ()
})
app.listen (5200)
```

Output:

On / page

[{"name":"abc","age":28},{"name":"def","age":40},{"name":"xyz","age":50}]

On /sort page

xyz = 50
def = 40
abc = 28

Route parameters

- ✓ Route parameters are named URL segments that are used to capture the values specified at their position in the URL.
- ✓ The captured values are populated in the **req.params** object, with the name of the route parameter specified in the path as their respective keys.

Route path: /calendar/:day/event/:ename
Request URL: http://localhost:6001/calendar/monday/event/birthday
req.params: {"day":"monday","ename":"birthday"}

To define routes with route parameters, simply specify the route parameters in the path of the route as shown below.

```
var express = require("express");
var app = express();
app.get('/calendar/:day/event/:ename', (req, res) => {
   res.send(req.params);
  });
app.listen (6001,()=>
{
   console.log ("server started");
})
```

To check: http://localhost:6001/calendar/monday/event/birthday

Output:

```
{"day":"monday","ename":"birthday"}
```

Query Parameters/req.query/req.body

- ✓ Express.js is a web application framework for Node.js that is used to build web servers and APIs. When a client sends data to the server, Express provides different ways to access that data. The most common methods are through query parameters and the request body. These are accessed using req.query and req.body respectively.

Query parameters are key value pairs that are sent in the URL after a question mark. They are mainly used to send small amounts of data such as filters, search terms, or sorting options.

Example of a URL with query parameters
http://localhost:3000/products?category=mobile&price=20000

In this example, category and price are query parameters.
In Express.js, query parameters are accessed using **req.query**.

req.query

It is an object in Express.js that contains all query parameters sent in the URL.

Example
If the request URL is
/data?name=rahul&age=25
Then req.query will be
{ name: "rahul", age: "25" }

This object allows developers to easily access and use query data in applications.

```
app.get('/data', (req, res) => {
const name = req.query.name;
const age = req.query.age;
res.send("Name: " + name + ", Age: " + age);
});
```

Characteristics of query parameters

- ✓ Query parameters are always received as strings
- ✓ They are visible in the URL
- ✓ They are mostly used with GET requests
- ✓ They are suitable for small amounts of data

req.body

The request body is used to send data from the client to the server in HTTP requests such as POST, PUT. This data is not visible in the URL and can include large or structured data such as JSON.

In Express.js, this data is accessed using **req.body**.

Example

```
app.use(express.urlencoded({ extended: true }));

app.post('/users', (req, res) => {
const name = req.body.name;
const age = req.body.age;
res.send("Name: " + name + ", Age: " + age);
});
```

Middleware requirement

To use req.body, middleware must be enabled. Express provides built in middleware such as express.json and express.urlencoded to parse incoming data. Without this middleware, req.body will be undefined.

Example

```
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
```

extended: false

- Only supports **simple (flat) data**
- Cannot handle nested objects
- Example → { "user": "test" }
- Uses Node.js built-in **querystring** module

extended: true

- Supports **nested objects & arrays**
- Cleaner and more useful structure
- Uses external **qs** library
- Example → { user: { name: "test" } }

Characteristics of req.body

✓ Data is not visible in the URL
✓ It can handle large and complex data
✓ Used mainly with POST, PUT methods
✓ Supports JSON and form data

Feature	express.json()	express.urlencoded()
Data type	JSON data	Form data (URL-encoded)
Example input	{ "name": "test" }	name=test&age=25
Parses into	req.body object	req.body object
Use case	APIs / backend services	Traditional web forms

Example

Create a web server that:

- **Accepts a GET request with a dynamic user ID in the URL**
- **Also accepts additional data (name, age) using query parameters**
- **Extracts:**
  - **id from route parameters**
  - **name and age from query string**
- **Returns a JSON response containing all received data**

```
const express = require("express");
const app = express();

// Route with params + query
app.get("/user/:id", (req, res) => {
   const userId = req.params.id;     // from URL path
   const name = req.query.name;      // from query string
   const age = req.query.age;

   res.json({
      message: "Data received",
      params: { id: userId },
      query: { name, age }
   });
});

app.listen(3005, () => {
   console.log("Server running on port 3005");
});
```

Write this in browser: http://localhost:3000/user/44?name=test&age=23

Output

```
{"message":"Data received","params":{"id":"44"},"query":{"name":"test","age":"23"}}
```

Example

Create a basic web application that:

- **Displays a form on the homepage to enter a user's name**
- **Sends the form data to the server using a get request**
- **The server should read the submitted name from the query string**
- **Finally, display a message confirming:**

```
const express = require("express");
const app = express();

// Homepage with form
app.get("/", (req, res) => {
  res.send(`
    <h1>User Form</h1>
    <h3>Create User</h3>
    <form action="/user" method="get">
      <input type="text" name="name" placeholder="Enter name" />
      <button type="submit">Create</button>
    </form>
  `);
});
// POST request
app.get("/user", (req, res) => {
  const name = req.query.name;
  res.send("User created with name " + name);
});
// Start server
app.listen(5504, () => {
  console.log("Server started on port 5504");
});
```

Example

Create a basic web application that:

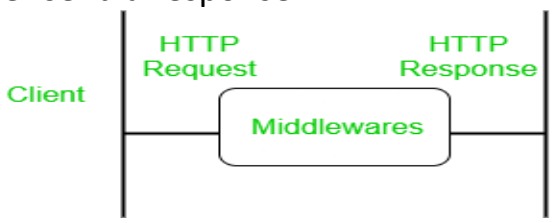
- **Displays a form on the homepage to enter a user's name**
- **Sends the form data to the server using a POST request**
- **The server should read the submitted name from the request body**
- **Finally, display a message confirming:**

```
const express = require("express");
const app = express();
// Middleware
app.use(express.urlencoded({ extended: true }));
```

```
// Homepage with form
app.get("/", (req, res) => {
  res.send(`
    <h1>User Form</h1>
    <h3>Create User</h3>
    <form action="/user" method="POST">
      <input type="text" name="name" placeholder="Enter name" />
      <button type="submit">Create</button>
    </form>
  `);
});
// POST request
app.post("/user", (req, res) => {
  const name = req.body.name;
  res.send("User created with name " + name);
});
// Start server
app.listen(5504, () => {
  console.log("Server started on port 5504");
});
```

Express Middleware

Express.js is a routing and Middleware framework for handling the different routing of the webpage and it works between the request and response cycle. Middleware gets executed after the server receives the request and before the controller actions send the response. Middleware has the access to the request object, responses object, and next, it can process the request before the server send a response.



An Express-based application is a series of middleware function calls.

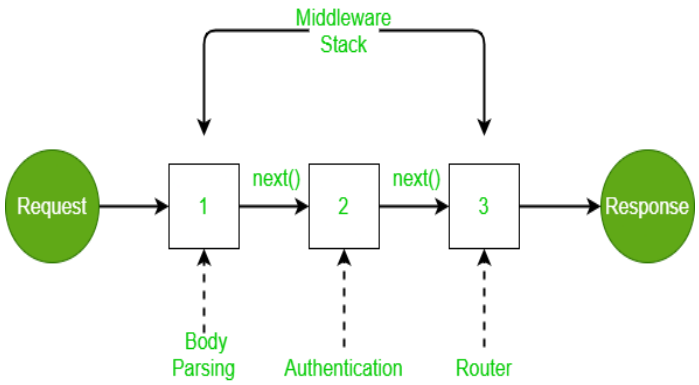
Advantages of using middleware:

- Middleware can process request objects multiple times before the server works for that request.
- Middleware can be used to add logging and authentication functionality.
- Middleware is used for setting some specific HTTP headers.
- Middleware helps for Optimization and better performance.

Middleware Chaining: Middleware can be chained from one to another, Hence creating a chain of functions that are executed in order. The last function sends the response back to the browser. So, before sending the response back to the browser the different middleware process the request.

The next() function in the express is responsible for calling the next middleware function if there is one.

Modified requests will be available to each middleware via the next function –



Middleware chaining example

In this case, the incoming request is modified and various operations are performed using several middlewares, and middleware is chained using the next function. The router sends the response back to the browser.

Middleware Syntax

```
const middlewareName = (req, res, next) => {
   // process request
   next();
};
```

Using Middleware

```
app.use(middlewareName);
```

OR

```
app.get("/route", middlewareName, (req, res) => {
   res.send("Response");
});
```

- req → request data
- res → response
- next() → move to next middleware

Middleware is a function with (req, res, next) that processes request before response.

Examples

Create an application using Express.js where:

- **Multiple middleware functions add student name, college, and marks**
- **Data is passed using req**
- **Final route displays all values**

```
const express = require("express");
const app = express();

// Middleware 1: Add student name
const addName = (req, res, next) => {
   req.name = "Samarth";
   console.log("Name added");
   next();
};

// Middleware 2: Add college name
const addCollege = (req, res, next) => {
   req.college = "LJ University";
   console.log("College added");
   next();
};

// Middleware 3: Perform calculation
const addMarks = (req, res, next) => {
   req.total = 50 + 40;
   console.log("Marks calculated");
   next();
};

// Route with middleware chain
app.get("/student", addName, addCollege, addMarks, (req, res) => {
   res.send(
      "Student Name: " + req.name +
      "<br>College: " + req.college +
      "<br>Total Marks: " + req.total
   );
});
```

```
// Start server
app.listen(6001, () => {
   console.log("Server running on port 6001");
});
```

Type in browser: http://localhost:6001/student

Output:

Browser:

Student Name: Samarth

College: LJ University

Total Marks: 90

Console:

Server running on port 6001

Name added

College added

Marks calculated

Develop a web application that simulates a college classroom entry system using middleware:

- **When a student accesses the /class route:**
  1. **Log that the student entered the campus**
  2. **Verify whether the student has a valid ID card**
  3. **If valid → allow entry and display a welcome message**
  4. **If not valid → deny access**

```
const express = require("express");
const app = express();

// Middleware 1: Entry Log (like college gate entry)
const entryLog = (req, res, next) => {
   console.log("Student entered campus");
   next();
};

// Middleware 2: ID Check
const checkID = (req, res, next) => {
   const hasID = true; // change to false to test

   if (hasID) {
      req.student = "Dhriti"; // add student name
      console.log("ID verified");
      next();
   } else {
      res.send("Access Denied: No ID Card");
   }
};

//  Apply middleware chain
app.use("/class", entryLog, checkID);

//  Final Route (Teacher)
app.get("/class/data", (req, res) => {
   res.send(`Welcome ${req.student} to the class`);
});

// Start server
```

```
app.listen(3000, () => {
   console.log("Server running on port 3000");
});
```

In browser: http://localhost:3000/class/data

Output:

Browser:

Welcome Dhriti to the class

Console:

Server running on port 3000

Student entered campus

ID verified

- Middleware function must have **(req, res, next)**
- Must call **next()** to move to next middleware
- If next() is not called → request will stop
- If response is sent using **res.send()**
  - Then do **NOT call next() after that**
- Middleware must be **defined before route**

```
app.use("/class", entryLog, checkID); // first
app.get("/class/data", ...);          // then
```

- Route must send final response

```
res.send(...)
```

app.use() handles all HTTP methods, while app.get() handles only GET requests, and middleware parameters must follow (req, res, next) order.

Middleware mounted on /class does **not require an exact match**. It runs for **any path that starts with /class**.

app.use("/class", ...) applies middleware to **all routes whose path begins with /class (prefix match, not exact match)**

here, app.get("/class/data", ...);